SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint Based Problem Solving

Course Code:	**ITA05**	**Course Title:**	**Computer Vision**
CO Assessed:	**CO5 – Naturalization (independent application using OpenCV)P5**	**Assessment:**	**Assessment Tool 1 – Constraint Based Problem Solving**
Weightage:	**25% of CIA**	**Total Marks:**	**100**
CO4 Statement:	*Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)*		
Student Name:	Malli Chandana	**Reg. No.:**	192472250
Section / Batch:	CSE-AI/2024-2028	**Date:**	24-08-2026

Code of Conduct

I Malli Chandana(192472250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:Malli Chandana

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	**25%**	**Clearly defines the problem and identifies all relevant constraints accurately**	**Identifies most constraints with minor gaps**	**Partial understanding; misses key constraints**	**Fails to identify constraints correctly**	
Constraint Analysis	**25%**	**Analyzes constraints effectively and prioritizes them logically**	**Provides reasonable analysis with some prioritization**	**Limited analysis; weak prioritization**	**No meaningful analysis or prioritization**	
Solution Development	**25%**	**Develops optimal and feasible solutions satisfying all constraints**	**Develops workable solutions with minor limitations**	**Solutions partially satisfy constraints**	**Solutions do not meet constraints**	
Application of Concepts	**15%**	**Effectively applies appropriate concepts, methods, or tools**	**Applies concepts with minor errors**	**Limited application; requires guidance**	**Unable to apply relevant concepts**	
Justification	**10%**	**Provides strong justification for decisions with logical reasoning**	**Provides justification with some clarity**	**Weak or unclear justification**	**No justification provided**	

REAL-TIME EDGE DETECTION SYSTEM
A. Capture Video
B. Detect Edges in Real Time
C. Display Results Efficiently

1. Introduction

Edge detection is a fundamental operation in computer vision. It identifies locations in an image where the intensity or color changes significantly. These locations often correspond to object boundaries, contours, surface changes, or important structural features. By converting a complex image into a set of meaningful boundaries, edge detection reduces visual information while preserving important geometric structure.
A real-time edge detection system extends this idea from still images to a continuous video stream. The system receives frames from a camera, processes each frame quickly, extracts edges, and displays the result with minimal delay. The major challenge is not only obtaining accurate edges but also completing the processing before the next frame arrives.

2. Problem Statement

The proposed system must capture live video, detect edges continuously, and display the processed output efficiently. A successful design should balance edge quality, processing speed, memory use, and responsiveness. Excessive computation can cause frame drops and delay, whereas excessive simplification can reduce the usefulness of the detected edges.

3. Objectives

- Capture frames continuously from a webcam or video source.
- Convert frames into a computationally efficient representation.
- Apply a robust edge detection method such as Canny.
- Display the original and edge-detected frames in real time.
- Reduce latency and unnecessary computation.
- Release camera and display resources safely when processing stops.

4. System Overview

The system is organized as a continuous processing pipeline. A frame is acquired from the camera, preprocessed, passed through an edge detector, and then displayed. The same sequence repeats for every frame until the user exits the application.

Figure 1. Real-Time Edge Detection Pipeline

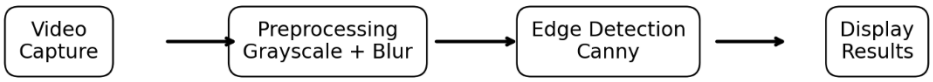

Figure 1. Real-Time Edge Detection Pipeline

4.1 Video Capture

Video capture is the first stage. In Python/OpenCV, VideoCapture can access a webcam or a stored video. Each iteration attempts to read one frame. The implementation should verify that the frame was successfully received before processing it. Camera resolution and frame rate are important because they directly affect the amount of data processed per second.

4.2 Preprocessing

The captured frame is normally converted from BGR color to grayscale. A grayscale image contains one intensity channel instead of three color channels, reducing memory traffic and computation. A small Gaussian blur can then be applied to suppress sensor noise and tiny intensity variations that could otherwise produce false edges.

4.3 Edge Detection

The Canny edge detector is suitable for this application because it produces relatively thin and well-localized edges. It combines smoothing, gradient calculation, non-maximum suppression, and hysteresis thresholding into a structured procedure.

4.4 Display

The processed frame can be shown using OpenCV's imshow function. The display loop should use a short waitKey interval so that the window remains responsive while the next frame is processed.

5. System Architecture

The architecture separates acquisition, buffering, processing, and visualization. This separation makes it easier to identify performance bottlenecks and replace individual components without redesigning the entire system.

Figure 5. System Architecture

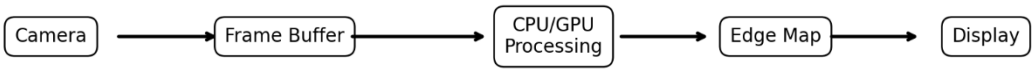

Figure 2. System Architecture

5.1 Functional Components

1. Camera/Input: supplies the live video stream.
2. Frame acquisition module: reads the next frame and checks capture status.
3. Preprocessing module: converts the frame to grayscale and optionally filters noise.
4. Edge detection module: applies Canny or another edge detector.
5. Display module: renders the original and processed results.
6. Control module: monitors keyboard input and handles termination.
7. Resource manager: releases the camera and closes windows safely.

5.2 Data Flow

The data flow is frame-based. Each frame follows the sequence Capture → Preprocess → Detect → Display. If one stage takes too long, the total per-frame latency increases. Therefore, optimization should focus on the most expensive operations and on avoiding work that does not contribute to the final output.

5.3 Real-Time Requirement

A practical target is to process frames at a rate close to the camera's useful frame rate. For example, at 30 FPS, a new frame is available approximately every 33 milliseconds. The processing pipeline should ideally finish within this budget. Actual performance depends on camera resolution, CPU/GPU capability, algorithm settings, and display overhead.

6. Edge Detection Methodology

6.1 Why Canny Edge Detection?

Canny is widely used because it provides good localization, relatively thin edges, and controllable sensitivity through two thresholds. It is more robust than simply applying a single intensity threshold because it uses gradient information and connects meaningful edge segments through hysteresis.

Figure 2. Main Stages of the Canny Edge Detector

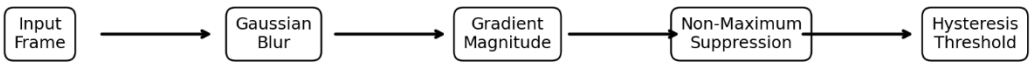

Figure 3. Main Stages of the Canny Edge Detector

1. Gaussian smoothing reduces high-frequency noise.
2. Gradient calculation estimates the magnitude and direction of intensity change.
3. Non-maximum suppression removes pixels that are not local maxima along the gradient direction.
4. Double thresholding classifies strong and weak candidate edges.
5. Hysteresis connects weak edges to strong edges when they form meaningful continuous boundaries.

6.3 Threshold Selection

Two thresholds are used in the Canny operation. The lower threshold controls the acceptance of weaker candidate edges, while the higher threshold identifies strong edges. Low thresholds can increase sensitivity but may introduce noise. High thresholds reduce noise but can miss subtle boundaries. The values should therefore be selected according to lighting, camera quality, and the intended application.

6.4 Alternative Edge Detectors

- Sobel: useful for computing directional gradients and relatively simple to implement.
- Laplacian: detects rapid intensity changes but can be more sensitive to noise.
- Scharr: improves gradient accuracy for certain small-kernel situations.
- Canny: generally preferred when clean, connected edges are required.

7. Implementation Using Python and OpenCV

The following implementation demonstrates the complete real-time pipeline. It captures webcam frames, converts them to grayscale, applies Gaussian smoothing, detects edges using Canny, and displays the original and edge images.

```
import cv2

cap = cv2.VideoCapture(0)
```

```
if not cap.isOpened():
    print("Error: Could not open camera")
    exit()

while True:
    ret, frame = cap.read()

    if not ret:
        print("Error: Could not read frame")
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    blur = cv2.GaussianBlur(gray, (5, 5), 0)

    edges = cv2.Canny(blur, 50, 150)

    cv2.imshow("Original", frame)
    cv2.imshow("Edges", edges)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

7.1 Explanation of the Program

- VideoCapture(0) opens the default camera.
- read() obtains the next frame and returns a success flag.
- cvtColor() converts the BGR frame to grayscale.
- GaussianBlur() suppresses small noise before edge extraction.
- Canny() produces the edge map.
- imshow() displays the original and processed frames.
- waitKey(1) allows the display to update while keeping the loop responsive.
- Pressing q exits the loop, after which release() and destroyAllWindows() free resources.

7.2 Algorithm

6. Start the application.
7. Initialize the camera.
8. Read a frame.
9. Check whether the frame is valid.

10. Convert the frame to grayscale.
11. Apply Gaussian filtering.
12. Run Canny edge detection.
13. Display the results.
14. Repeat until the exit key is pressed.
15. Release resources and terminate.

8. Results and Performance Evaluation

The output of the system consists of the live camera frame and an edge map. The edge map highlights boundaries such as object outlines, furniture, road markings, people, and other intensity transitions. The exact appearance depends on lighting, camera focus, motion, and threshold settings.

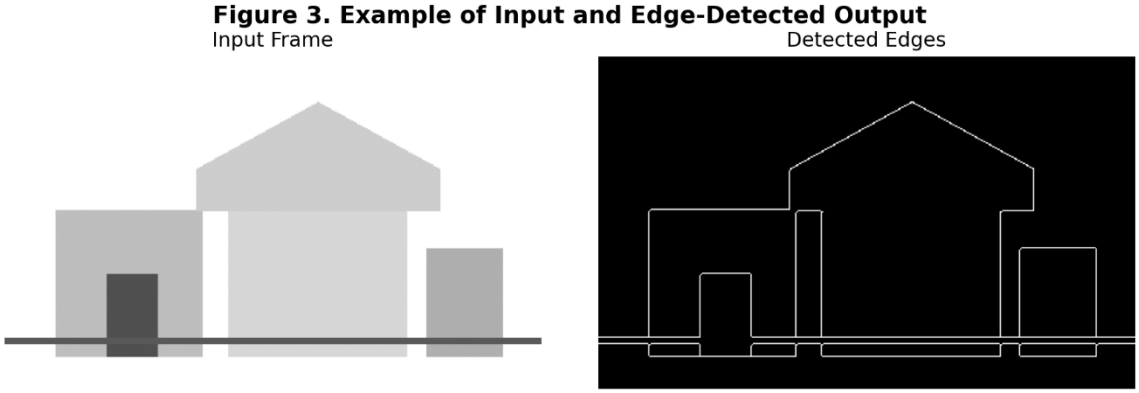

Figure 4. Example of Input and Edge-Detected Output

8.1 Expected Results

- Edges should appear clearly around major object boundaries.
- Noise should be reduced by grayscale conversion and Gaussian smoothing.
- The display should update continuously without obvious freezing.
- The camera feed should remain synchronized closely enough with the processed result for real-time observation.

8.2 Performance Metrics

The most useful performance measures are frames per second (FPS), end-to-end latency, CPU/GPU utilization, memory consumption, and edge quality. FPS describes throughput, while latency measures how long it takes a captured frame to become a displayed result. A system can have acceptable FPS but still feel unresponsive if buffering creates excessive delay.

8.3 Factors Affecting Performance

- Frame resolution: larger images require more pixels to process.
- Camera frame rate: higher FPS reduces the available processing time per frame.

- Gaussian kernel size: larger kernels increase computation.
- Display operations: rendering multiple large windows adds overhead.
- Computer hardware: CPU speed, memory bandwidth, and available GPU acceleration influence throughput.
- Background applications: other processes can compete for CPU and memory resources.

9. Optimization Strategies

Optimization is essential because real-time computer vision is constrained by a time budget for every frame. The goal is to reduce unnecessary work while preserving sufficient edge quality.

9.1 Convert to Grayscale Early

Canny operates on a single-channel intensity image. Converting the frame to grayscale reduces the amount of data compared with processing three color channels. This is one of the simplest and most effective optimizations.

9.2 Use an Appropriate Resolution

Processing a 1920×1080 frame requires far more work than processing a 640×480 frame. If very fine spatial detail is not necessary, reducing the camera resolution can substantially improve FPS and lower latency.

9.3 Choose a Small Gaussian Kernel

A 5×5 Gaussian kernel is a common compromise between smoothing and speed. Increasing the kernel size may remove more noise but also increases computation and can blur small useful structures.

9.4 Region of Interest

If the application is interested only in a particular portion of the scene, process that region instead of the complete frame. For example, a road-monitoring system can restrict processing to the roadway area.

9.5 Avoid Unnecessary Operations

Do not repeatedly resize, copy, or transform frames unless required. Keep the pipeline simple and perform only operations that directly contribute to the result.

9.6 Efficient Display

Use a short waitKey interval and avoid creating new windows inside the processing loop. Reusing existing windows reduces overhead and keeps the user interface responsive.

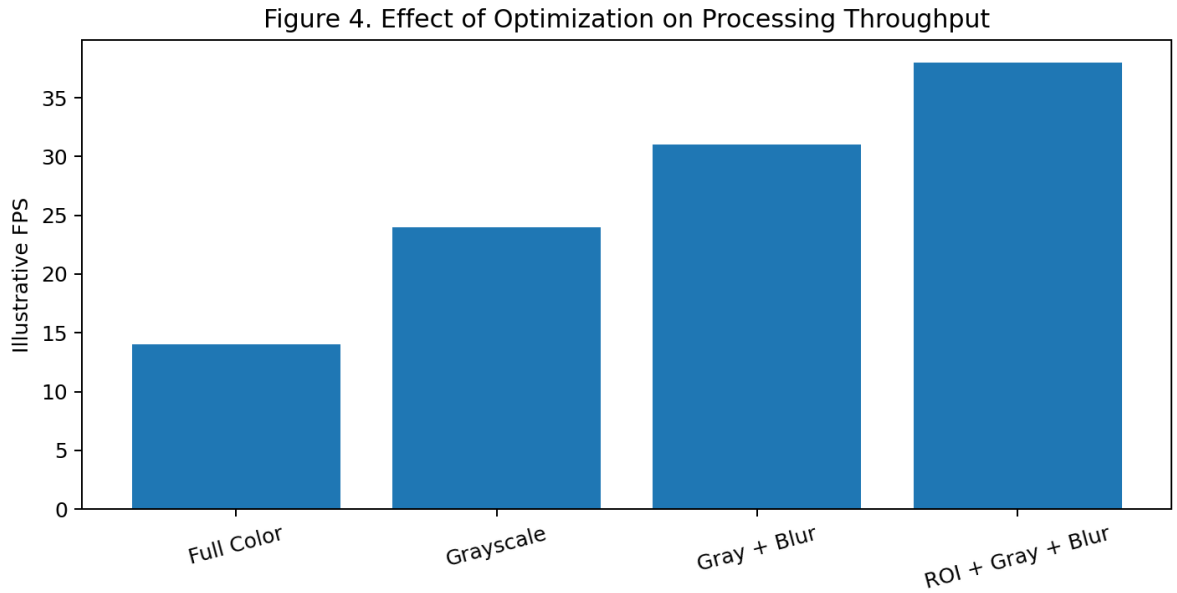

Figure 5. Illustrative Effect of Optimization on Processing Throughput

10. Advanced Optimization Techniques

10.1 Frame Skipping

If the camera produces more frames than the processor can handle, selected frames can be skipped. This reduces workload but should be used carefully because excessive skipping can make motion appear less smooth.

10.2 Multithreading or Producer–Consumer Design

Camera acquisition and image processing can be separated into different execution stages. A capture thread can continuously obtain frames while a processing thread handles the most recent available frame. This can reduce waiting, although synchronization and buffer management must be designed carefully.

10.3 Hardware Acceleration

Where supported, computationally intensive operations can be moved to a GPU. OpenCV provides mechanisms for accelerated processing on compatible systems. Hardware acceleration is particularly useful when processing high-resolution video or when multiple vision algorithms are executed together.

10.4 Buffer Management

A long queue of old frames increases latency. For real-time applications, it is often preferable to process the newest frame rather than allowing an increasingly large backlog. This keeps the displayed result closer to the current camera view.

10.5 Region-Based Processing

A region of interest can be dynamically selected according to the application. In autonomous driving, for example, the lower portion of the frame may contain the road and can receive higher processing priority.

10.6 Parameter Tuning

Thresholds should be tuned experimentally. If the image contains substantial noise, smoothing and thresholds can be adjusted. If important weak edges are missing, the lower threshold may need to be reduced. Parameter tuning should be evaluated using both visual quality and FPS.

10.7 Optimization Trade-Off

Optimization is not simply about maximizing FPS. Reducing resolution or aggressively filtering the image can improve speed while decreasing edge quality. The best design is therefore the one that satisfies the application's minimum edge accuracy and maximum acceptable latency.

11. Applications

Real-time edge detection is useful wherever object boundaries or structural changes need to be identified quickly.

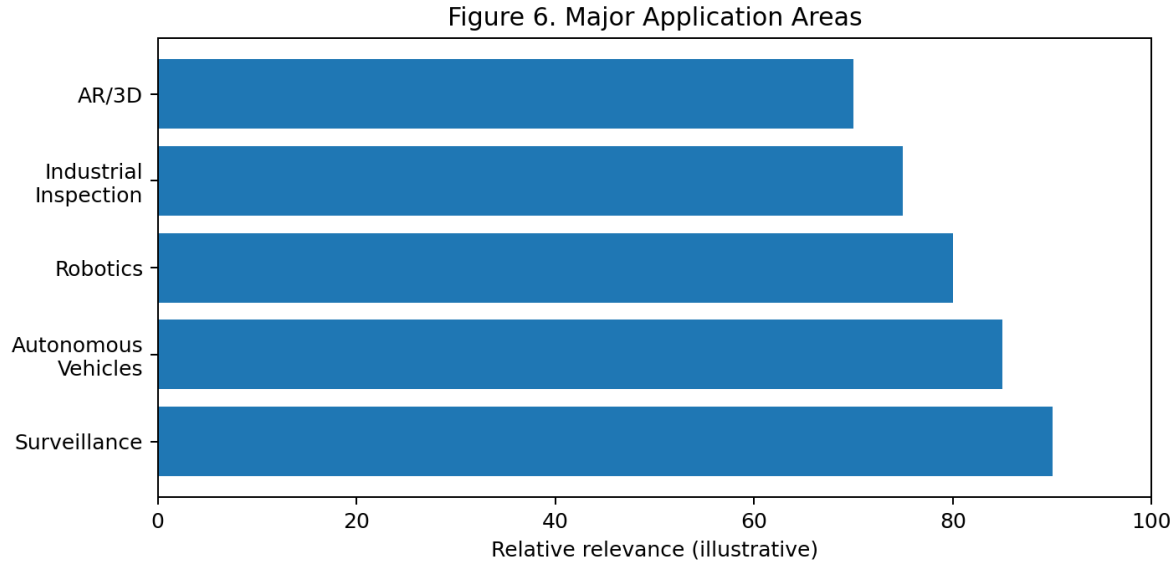

Figure 6. Major Application Areas

- Surveillance: outlines of people and objects can support motion and object-analysis systems.
- Autonomous vehicles: lane boundaries, road structures, and object contours can provide useful visual cues.
- Robotics: robots can use edge information to identify shapes, obstacles, and workspace boundaries.

- Industrial inspection: edges can reveal cracks, missing components, incorrect dimensions, or surface defects.
- Document analysis: text and page boundaries can be detected before further processing.
- Augmented reality and 3D vision: edge information can assist in identifying scene structure.
- Medical imaging: boundaries in X-rays, scans, and other images can support later segmentation tasks.

12. Advantages

- Simple and widely available computer-vision approach.
- Works with ordinary webcams.
- Can operate continuously with low latency when properly optimized.
- Produces visually interpretable results.
- Can be integrated with object detection, tracking, and segmentation systems.

13. Limitations

- Performance decreases as image resolution increases.
- Poor lighting can create weak or noisy edges.
- Moving objects may produce blurred boundaries.
- Canny thresholds often require application-specific tuning.
- Edge detection alone does not identify what an object is; it only highlights boundaries.

14. Conclusion

The Real-Time Edge Detection System provides a practical example of a continuous computer-vision pipeline. The system captures video, preprocesses each frame, detects edges using Canny, and displays the results with low delay. Grayscale conversion, suitable resolution, small Gaussian filtering, region-of-interest processing, efficient display, and careful buffer management are important optimization strategies. The design can serve as a foundation for more advanced systems such as lane detection, object tracking, robotics, surveillance, and industrial inspection.

15. References

- OpenCV Documentation – Image Processing and Video I/O modules.
- OpenCV Python tutorials – camera capture and image processing.
- J. Canny, “A Computational Approach to Edge Detection,” IEEE Transactions on Pattern Analysis and Machine Intelligence.
- R. C. Gonzalez and R. E. Woods, Digital Image Processing.
- Python Software Foundation, Python documentation.